

## Primer control de laboratorio

Crea un fichero que se llame “respuestas.txt” donde escribirás las respuestas y el código para los apartados de los ejercicios del control. Indica para cada respuesta, el **número de ejercicio y el número de apartado** (por ejemplo, 1.a, 1.b, ...).

**Importante:** para cada uno de los ejercicios tienes que partir del código suministrado de Zeos.

### 1. (6 puntos) Mailboxes?

Queremos añadir una nueva funcionalidad a ZeOS que permita enviar información entre 2 procesos: lector y escritor.

Tienes que añadir 2 nuevas llamadas a sistema:

```
int writeTo(int pid, char* src, int size);  
int readFrom(int* pid, char *dst);
```

Cada proceso tendrá una página para **enviar información** siempre mapeada en la dirección lógica 0x3FF000, así como un contador del número de bytes que se han enviado, y una lista de procesos bloqueados para escribir.

La función *writeTo* permite enviar *size* bytes desde *src* al proceso *pid*. Para ello debe, copiar la región de memoria a escribir en la dirección lógica 0x3FF000, quedarse bloqueado en la lista de procesos del proceso lector *pid* hasta que el *readFrom* lo desbloquee, momento que dará la escritura por finalizada. Si el proceso lector estaba bloqueado, debe desbloquearlo. Para simplificar, sólo permitimos escrituras de hasta 4096 bytes.

La función *readFrom* permite leer la información enviada por otros procesos. Si nadie ha escrito nada, el proceso actual debe bloquearse. Caso contrario, debe mapear la página física del primer proceso escritor en la dirección 0x3FD000, copiar tantos bytes como se hayan escrito a partir de la dirección *dst*, desbloquear al proceso que ha escrito y devolver su pid en el parámetro *pid*. La función, al acabar, debe desmapear la página y devolver el número de bytes leídos.

Estas llamadas deben implementarse usando el número de servicio 37 y 38 respectivamente. Puedes ignorar la gestión de errores de estas llamadas.

En particular, tienes que:

- (0.5 puntos) Añade los wrappers necesarios.
- (0.5 puntos) Define una estructura nueva con la información mínima necesaria por proceso.
- (0.5 puntos) Indica los cambios necesarios en las estructuras del sistema al añadir estas dos nuevas llamadas.
- (0.5 puntos) Implementa la función *initializeData(PCB)* per inicializar la estructura de un proceso dado.
- (0.5 puntos) Implementa la función *mapPhysicalPageFrom(PCB, address)* para mapear la página física del proceso en la dirección pasada.
- (0.5 puntos) Implementa una función *unmapPage(PCB, address)* que permita desmapear las páginas usadas en el *writeTo* y el *readFrom* respectivamente.
- (1 punto) Implementa *sys\_writeTo*.
- (1 punto) Implementa *sys\_readFrom*.
- (1 punto) Indica los cambios necesarios en las llamadas a sistema preexistentes.

## 2. (4 puntos) Task switch

El Dr. Baka Baka propone cambiar la implementación del *task\_switch* para hacerla como hasta ahora pero “*en alto nivel directamente*”:

```
void task_switch(struct task_struct* new) {
    save_regs();
    inner_task_switch(new);
    restore_regs();
}
```

Con la particularidad que las funciones *save\_regs* y *restore\_regs* guardaran y restauraran respectivamente los registros EBX, ESI y EDI en la pila de usuario.

- (0.5 puntos) Antes de nada, hay que desactivar la optimización en la compilación del fichero *sched.c* (que es donde añadiremos esta nueva función *task\_switch*). Indica las modificaciones necesarias en el Makefile.
- (1 punto) Crea 2 funciones en ensamblador *save\_regs* y *restore\_regs* en un fichero nuevo *exam.S* que se puedan llamar desde C, que cumplan las convenciones de llamadas en C (exceptuando el valor final de los registros pedidos) y que hagan **literalmente** lo que pide el Dr. Baka Baka. Indica las modificaciones en el Makefile así como el código necesario para las funciones.
- (0.5 puntos) Indica el desensamblado de la rutina *task\_switch* y cómo lo has hallado.

Tras probar estos cambios, te das cuenta de que no funciona, ya que el contenido de los registros varia después de invocar al cambio de contexto. Tras explicarle al Sr. Baka Baka que su idea no puede funcionar por que los valores guardados en la pila se pierden al salir de la función, te comenta muy tranquilo: “Claro, porque las convenciones de C te están limitando. Tienes que romperlas.”

- (1 punto) Implementa la función *save\_regs* tal y como quiere el Sr Baka Baka. En particular, al salir de la función, la pila debe contener los registros guardados y ESP apuntar al último valor.
- (1 punto) Implementa la función *restore\_regs* tal y como quiere el Sr Baka Baka. En particular, esta función espera tener en la pila los 3 registros guardados antes, tiene que restaurarlos y salir normalmente.

## 3. (1 punto) Generic Competences Third Language (Development level: mid)

The following paragraphs are written by Craig Marcho from Microsoft in 2019:

*“When a program is first loaded, the logical memory address range of the application is divided into fixed size units called pages. As each page is referenced by a program, it is mapped to a physical page that resides in physical memory. The mapping is dynamic which ensures that logical addresses that are frequently referenced reside in physical memory. Remember that each individual process that is launched is allocated its own virtual*

# SOA (14/11/2025)

---

*address space and application program threads are only permitted to directly access the virtual memory locations that are associated with their parent process' address space. This is where Page Tables come into play.*

*Page Tables are built for each process address space. The Page Table maps logical virtual addresses for a process to physical memory locations. The location for a set of Page Tables for a process is passed to the processor hardware during a context switch. The processor refers to the Page Tables to perform virtual to physical address translation as the process threads are executed."*

Create a text file named "generic.txt" and answer the following questions (since this competence is about text comprehension, you can answer in whatever language you prefer):

- 1.-Describe the page table structure of a process in ZeOS
- 2.-Does ZeOS and Windows present a similar mechanism to pass the location of the Page Table to the hardware?
- 3.- What is the difference between ZeOS and Windows when creating the Page Table for a process?

## 4. Entrega

Sube al Racó los ficheros "respuestas.txt" y "generic.txt" junto con el código que hayas creado en cada ejercicio. Para entregar el código utiliza:

```
> tar zcfv examen.tar.gz respuestas.txt generic.txt ejercicio1  
ejercicio2
```

# SOA (14/11/2025)

---

## SOLUCION

1.a

WRAPPERS normales con EAX=37 y 38 respectivamente

1.b

```
struct pipe {
    int numbytes;           //number of bytes in page
    struct list_head writers; //list of pending writers
}
```

1.c

Modificar sys\_call\_table añadiendo las entradas necesarias

En el PCB>

```
...
struct pipe pipeinfo;
...
```

1.d

```
void initializeData(struct task_struct*p) {
    set_ss_page(get_PT(p), 0x3FF, alloc_frame());
    p->pipeinfo.numbytes=0;
    INIT_LIST_HEAD(&p->pipeinfo.writers);
}
```

1.e

```
int mapPhysicalPageFrom(struct task_struct* p, int address) {
    int frame = get_frame(get_PT(p), 0x3FF);
    page_table_entry *myPT = get_PT(current());

    set_ss_page(myPT, address>>12, frame);
}
```

1.f

```
int unmapPage(struct task_struct*p, int address) {
    page_table_entry *myPT = get_PT(current());
    del_ss_page(myPT, address>>12);
    set_cr3(get_DIR(current()));
}
```

1.g

```
int sys_writeTo(int pid, char* src, int size) {
    struct task_struct*p = findTaskByPid(pid);
    copy_data(src, 0x3FF000, size);
    current()->pipeinfo.numbytes = size;

    if (p->state == BLOCKED) //Unblock READER
        update_process_state(p, &readyqueue);
    //BLOCK
    update_process_state(current(), &p->pipeinfo.writers);
    sched_next();
    return size;
}
```

1.h

```
int sys_recvFrom(int* pid, char* dst){
    if (list_empty(&current()->pipeinfo.writers)) {
        update_process_state(current(), &blockedq);
        sched_next()
    }
}
```

# SOA (14/11/2025)

---

```
t = list_first(&current()->pipeinfo.writers)
struct task_struct* p = list_head_2_task_struct(t),
int size = p -> pipeinfo.numbytes
mapPhysicalPageFrom(p, 0x3FD000);
copy_data(0x3FD000, dst, size);
*pid = t->pid; //copy_to_user(pid, &t->pid, sizeof(int));
//copied! unblock writer & finish
update_process_state(p, &ready)
unmapPage(current(), 0x3FD000);
return size;
}

1.i
init_task1
    initializeData(task1);
fork
    initalizeData(newchild);
exit
    free_frame(get_frame(get_PT(current()), 0x3FF));
    //unblock writers
    for_each_safe(p, tmp, current()->pipeinfo.writers) {
        list_del(p);
        update_process_state(list_head_2_task_struct(p),
&readyqueue);
    }

2.a
sched.o: sched.c
    gcc ... (sin -O2)

2.b
exam.S
ENTRY(saveRegs)
    pushl %ebp
    movl %esp, %ebp
    pushl %esi
    pushl %edi
    pushl %ebx
    movl %ebp, %esp
    popl %ebp
    ret
ENTRY(restoreRegs)          //Esta funcion NO cumple convención
(Pero hace literalmente lo que pide BakaBaka)
    pushl %ebp
    movl %esp, %ebp
    popl %ebx
    popl %edi
    popl %esi
    movl %ebp, %esp
    popl %ebp
    ret

Makefile    //añadir exam.o a los binarios de sistema
    SYSTEMOBS+= exam.o

2.c
objdump -d system|less
```

# SOA (14/11/2025)

---

2.d

```
//Esta rutina NO cumple convención porque modifica ESP
ENTRY(saveRegs)
    pushl %esi
    pushl %edi
    pushl %ebx
    pushl 12(%esp)    // Volvemos a apilar la @retorno
    ret
```

2.e

```
//Esta rutina NO cumple convención porque justamente modifica
EBX, ESI, EDI y ESP
ENTRY(restoreRegs)
    popl %eax    //Copiar @retorno a la posicion donde estaba
la @retorno de saveRegs
    movl %eax, 12(%esp)
    popl %ebx
    popl %edi
    popl %esi
    ret
```